

Programming Assignment 1

1 Introduction

In this assignment, you are asked to implement a simple queuing system. You will write two C programs—a server and a client—and experiment with them.

2 Description

The system consists of a single server and one or more clients. Clients generate jobs, send them to the server, which, in turn, processes them and records their statistics. The communication is carried out via UDP. A detailed description of the client and server follows.

2.1 Client

The clients generate jobs, whose arrival times are governed by Poisson process and whose lengths are exponentially distributed. When a client process is invoked, it receives, as command line arguments, the contact info of the server (IP address and port number), the number of jobs to generate, and parameters for the Poisson process and the exponential distribution. More specifically, a client is invoked with the following command line:

```
client ip port num_jobs seed lambda mu
```

where *ip* is the IP address of the server in dotted decimal notation, *port* is the port number of the server, *num_jobs* is the total number of jobs that will be generated by the client, *seed* is an integer used as a seed to the random number generator, *lambda* is a floating-point number used as the Poisson parameter, and *mu* is a floating-point number used as a parameter for the exponential distribution. The client writes a log (see below) to `stdout` which can be redirected to a file, as in the following example:

```
client 127.0.0.1 9000 128 211 50.0 20.0 > client.tsv
```

The client operates as follows. After parsing the command line arguments, the client initializes the seed using `srand()`. The client then does the following for each of the *num_jobs* jobs:

1. Sample two exponential random variables, say *x* and *y* with parameters *lambda* and *mu*, respectively. These are floating-point numbers representing time in milliseconds.
2. Sleep for $\lfloor 10^6 \cdot x \rfloor$ nanoseconds.
3. Send a UDP datagram to the server, describing the job attributes. The message contains the following values.
 - Client ID (32 bits, unique per client). For example, the process identifier.

- Job index (16 bits): the number of the job in that client, starting at zero (i.e., the first job index is 0).
 - Job length in nanoseconds (32 bits): this is $\lfloor 10^6 \cdot y \rfloor$.
4. Write a line to `stdout` the IP with port (in hexadecimal), the client ID, the job index, and the sampled values, separated by tabs. The line format should be similar to the generated by the following statement. See examples in the moodle site.

```
printf("%08x:%04x\t%d:%d\t%d\t%d\n", ip, port, id, index, floor_x, floor_y);
```

After generating all *num_jobs* jobs, the client exits.

Hints.

- Numbers from the command line can be parsed using `strtol()` and `strtod()`. IP addresses may be parsed using `inet_pton()`.
- Process identifiers (PID) can be retrieved using `getpid()`.
- Sleeping can be implemented using `nanosleep()`, or the C11 `thrd_sleep()`. These functions use `struct timespec` to represent time – double-check your results when converting to and from milliseconds.
- The following code can be used to sample an exponential random variable:

```
#include <stdlib.h>

double randexp(double lambda) {
    double u = rand() / ((double)RAND_MAX + 1.0);
    return -log(1.0 - u) / lambda;
}
```

2.2 Server

The server has two threads: an acceptor thread and a worker thread. The acceptor thread waits on the socket for messages. When a message arrives, the job it describes is inserted at the tail of a FIFO queue (if there is room). The worker thread, when idle, dequeues the first job from the head of the FIFO queue (if the queue is nonempty) and “executes” it: execution is simulated by sleeping for *y* milliseconds (that is, $\lfloor 10^6 \cdot y \rfloor$ nanoseconds), where *y* is obtained from the job structure. If the queue is empty when the server is done, the server waits. Specifically, the server is invoked using the following command line:

`server port num_jobs q-size`

where *port* is port number to which the server binds, *num_jobs* is the total number of jobs that will be processed by the server, and *q-size* is the maximum number of jobs the queue can contain. After receiving *num_jobs* jobs, the acceptor thread joins with the worker thread, waiting for it to finish processing pending jobs (if any), and then the process exits.

For each job processed, the worker thread writes to `stdout` the following values, separated by tabs:

- IP address and port number of the client (in hexadecimal)
- Job (client) ID

- Job index
- arrival since the server started running (in nanoseconds)
- departure time (when was the job finished) since the the server started running (in nanoseconds)
- number of jobs in the queue
- total current demand: the sum of all lengths of jobs in the queue, including the job currently executing.

Use the following formatting:

```
printf("%08x:%04x\t%d:%d\t%ld\t%ld\t%d\t%ld\n",
       ip, port, job.id, job.index, arrival_time, departure_time, q_num, q_time);
```

See example files in the moodle site.

Note that you are asked to implement the *drop tail* policy: when a job arrives at a full queue, it is dropped. However, dropped jobs *are* counted as jobs accepted by the server, even though they do not enter the queue.

Hints.

- Since both threads access the queue, some synchronization, such as a mutex is required.
- Waiting for the queue to be nonempty by the worker thread can be implemented by a condition variable.
- To get the current time, use `clock_gettime()` with `CLOCK_MONOTONIC`. Do not use other functions/syscalls to get the time.
- In the computation of the elapsed time in nanoseconds, use signed arithmetic.

Recall that using UDP, the server must still `bind()` to its address, but does not need to call `listen()` or `accept()`, and instead calls `recvfrom()`. Similarly, the client does not need to `connect()`, and instead calls `sendto()`. The socket is created using `socket(AF_INET, SOCK_DGRAM, 0)`.

2.3 Software requirements

The program should be memory-safe. Hazards include memory leaks, memory corruption, buffer overflows, etc. To aid with development, compile with `-Wall -Wextra -Wpedantic` flags. We suggest using tools like Valgrind¹ and AddressSanitizer². The programs should be clean using these tools. We also require strict error handling. All errors should be logged properly to `stderr` (e.g., using `perror`), and exited with a non-zero error code. Even on an error, the program should not leak memory and all sockets should be closed. When using dynamic memory allocation, always check the result of `malloc`. You can assume that the command-line arguments are correct.

Message format. Think about the way values are written and retrieved from the datagrams. You may want to use standard conversion functions such as `ntohl()`.

¹Also add compiler flags `-g -mno-avx2`

²Just add compiler flags `-fsanitize=address -fsanitize=pointer-compare -fsanitize=pointer-subtract`

3 Experiments

Run the following experiments and submit their results (see Section 5).

1. Single client, unbounded queue: This experiment simulates the basic M/M/1 system. The unbounded buffer is guaranteed by using queue size that is larger than *num_jobs*. Run experiments with the following parameters.
 - $(\mu, \lambda) = (5, 3)$, $(\mu, \lambda) = (3, 5)$ and $(\mu, \lambda) = (50, 30)$. Run with 1000 jobs and with 4000 jobs.
 - With 2000 jobs: $(\mu, \lambda) = (50, 35); (50, 40); (50, 45)$
2. Two clients, unbounded queue: start two clients, each generating 2000 jobs, with $(\mu, \lambda) = (50, 20)$. Multiple clients in the same machine can be started in the command line using `&`. (Try clients on different machines as well!)
3. Single client, bounded queue: Limit the queue to hold 10 jobs and run the system with 2000 jobs and $(\mu, \lambda) = (50, 45); (50, 48)$.

For each of the above experiments, compute the following statistics:

- Average and median job time (the time since arrival until departure, i.e., the end of processing by the server).
- Average and median queue occupancy (number of jobs in the queue)
- For the bounded queue scenarios only: compute the total and percentage of the number of jobs dropped.

4 Documentation

Inline documentation. Each function should have at least one documenting comment that describes its input (arguments and global variables), output (return value and side effects), and action at a high level. If the function is long, sub-descriptions should be present too.

README file. This file, in PDF format, must include the following.

- Students' names, student numbers, and email addresses.
- Submission contents (file descriptions).
- Results: For each experiment, provide the following.
 1. All statistics computed (see Section 3),
 2. a chart of the queue size over time, and
 3. a histogram of the job system times (time from arrival until departure), in 10 columns.

To generate these graphics from the server logs, use your favorite tool, e.g., Python, Excel, Google Sheets, or an AI agent. See examples in the moodle site.

5 Submission

Files to submit. You should submit (in the moodle site) one zip file, which should be named PA1_ID1_ID2.zip, replacing ID1 and ID2 with the IDs of the pair. The zip file should contain: (i) `server.c` and `client.c` (ii) `Makefile` (iii) `README.pdf`. The `Makefile` may look like this:

```
CC= gcc # Clang is another alternative
CFLAGS= -Wall -Wextra -O2 -std=c11 -Wpedantic -march=native
# using gcc version 14.2
all: server client
.PHONY: all
server: server.c
    $(CC) $(CFLAGS) -o server server.c
client: client.c
    $(CC) $(CFLAGS) -o client client.c -lm
```

5.1 Testing environment

We will test your code on Ubuntu 22.04 LTS. If you are running GNU/Linux or a BSD (like MacOS), it is probably fine. If you are running Windows, we recommend installing Windows Subsystem for Linux (WSL). Otherwise, use VirtualBox installed on machines in the computer lab (see the moodle site). For an additional safety check if you are running a different system, try to compile in <https://godbolt.org>, which has a similar setup.

5.2 Grading

Grading will primarily focus on correctness. We distinguish between the following levels of severity:

1. Socket/protocol error: Errors like using the wrong protocol, or using the wrong syscalls.
2. Output error: Wrong units, missing column, integer overflow, etc.
3. Safety error: Memory leaks, memory unsafe operations, open sockets, unsafely accessing shared data from different threads, etc.
4. Error handling: not handling an error or handling it incorrectly (e.g., one of the mistakes above).
5. “Technically wrong, but works”: There is a conceptual mistake on the code that never triggers. For example, forgetting to use `htons`.

6 Notes about implementation

Using libraries. You can use any C standard until (and including) C23 (specify in the `Makefile`).

We only allow using headers and libraries that come packaged with most Linux/BSD distributions. There are standard libraries such as `libc` and system libraries (such as those under `sys`) that you do not need to specify. There are other libraries that we allow and need to be linked in the `Makefile` script.

These include: (i) `libm`, (ii) `libatomic`, (iii) `pthread`. As a suggestion, instead of using `pthread`, the C11 concurrency support library should suffice for our uses.

Using C++. While you are allowed to use C++, you must guarantee that exceptions are handled correctly and do not leak. If you choose to use other C++ features, such as RAII, we will grade correctness accordingly. You can use any C++ standard until (and including) C++23 (specify in the Makefile).

Using LLMs. The use of LLMs to generate code is allowed, with proper attribution, review, and documentation.

7 Tips and tricks

System libraries. C is notorious for not having common data-structures. However, system headers (that is, `<sys/...>`) are installed in Unix systems and have basic support for many data-structures. Relevant for us is the `<sys/queue.h>` header that has macros to quickly implement a queue. See the man pages for `TAILQ` and `STAILQ` for examples. These are based on linked-lists and are not bounded.

Debugging. If you have found memory bugs (e.g., found by Valgrind), you should recompile with `-Og -ggdb3` flags instead of `-O2` and use GDB, the GNU debugger.

POSIX. Some functions are hidden POSIX, thus they require in the preamble (before the imports) a defined symbol such as `#define _GNU_SOURCE` or `#define _POSIX_C_SOURCE 200809L`. Consult the manual and make sure that `-Wpedantic` does not complain.